

Ghost: Migration to GKE

Moving a customer-facing blog from on-premise to Google Cloud, across Test, Acceptance, and Production — with automated, end-to-end deployment.

Prepared for: CTO review

Status: Design complete, MVP running, code-reviewable infrastructure



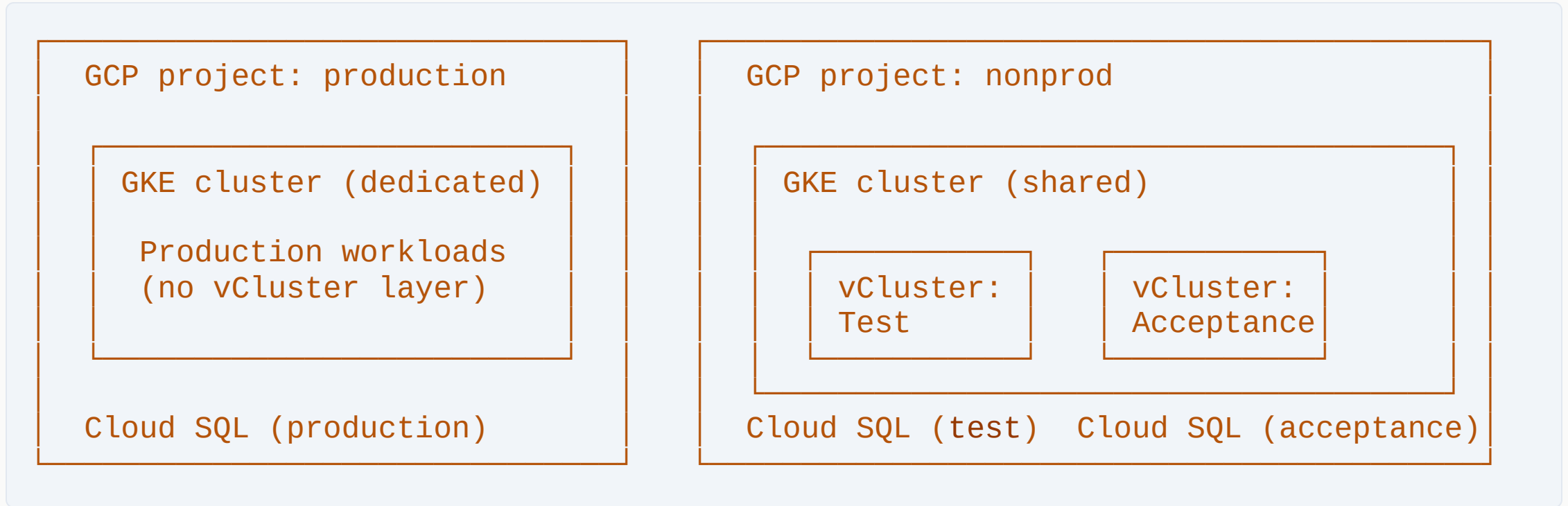
The ask, and our approach

Today: Ghost + NGINX reverse proxy + MySQL cluster, on-premise, three environments.

Requirements: Kubernetes as a Service · external DB + persistent storage · secure exposure · fully automated deployment · metrics/logs · infrastructure as code, one codebase.

Our approach, one sentence: GKE Autopilot, sized to risk — full isolation for Production, cost-efficient sharing for Test/Acceptance — with Terraform provisioning infrastructure and Flux delivering the application, so a git push is the entire deploy action.

High-level design



Two GCP projects, two GKE clusters. Full detail: [docs/ARCHITECTURE.md](#) .

Environment strategy: isolation matched to risk

	Production	Test / Acceptance
Cluster	Dedicated GKE, own project	Shared GKE, own project
Isolation	Full physical isolation	vCluster: own API server, RBAC, CRDs — virtual
Database	Own Cloud SQL instance	Own Cloud SQL instance each
Cost	Full cluster cost	One cluster, shared

Why this split: every additional real cluster carries its own cost, even mostly idle. Production is customer-facing, so it gets full isolation. Test and Acceptance don't carry that risk, so they share compute — isolated from each other at the Kubernetes control-plane level, not by paying for a third physical cluster.

Stated plainly: vCluster virtualizes the control plane, not the kernel — anyone with `nonprod` project access could, in principle, reach both at the node level. Accepted because neither is customer-facing, and it's exactly why Production doesn't use this pattern.

Automation: two engines, no manual step

Terraform (via Cloud Build) → provisions infrastructure
Flux → delivers the application

- **Infrastructure changes:** a PR, a reviewed plan, an apply — Production changes require a deliberate manual approval, everything else auto-applies.
- **Application changes:** push to `main` → Test deploys immediately. Cut a tag → Acceptance deploys. A `CODEOWNERS`-gated PR bumping one pinned value → Production deploys.
- **No CI runner performs a deploy anywhere in this design** — git state is the deploy mechanism, for both infrastructure and application.

Detail: `docs/AUTOMATION.md` , `docs/BRANCHING.md` .

Security by design

- **No credentials in pods.** Ghost authenticates to Cloud SQL and Secret Manager via Workload Identity — no service account key files, anywhere.
- **Database has no public IP.** Private Service Access, reachable only from inside its own VPC.
- **Least-privilege automation.** Every Terraform stage has its own narrowly-scoped service account — never `owner` / `editor` .
- **Secrets never committed in plaintext** — encrypted with SOPS in the demo, bridged from Secret Manager on real GKE.

Observability

- **Built in, by default:** GKE Autopilot ships every pod's logs and baseline metrics to Cloud Logging/Monitoring — nothing extra to install.
- **Layered on top:** a self-hosted Prometheus/Loki/Grafana stack, with a Ghost-specific dashboard (pod health, resource usage, live logs).
- **Pipeline alerting:** every deploy success/failure posts to Slack automatically — a broken promotion is visible immediately, not discovered later.

Detail: [docs/MONITORING.md](#) .

Requirements coverage

Requirement	Status
Kubernetes as a Service	✓ GKE Autopilot
External DB + persistent volumes	✓ Cloud SQL + Filestore
Secure exposure	✓ Gateway API + TLS everywhere
Automated end-to-end deployment	✓ Terraform + Flux, no manual step
Metrics and logs	✓ Cloud Logging/Monitoring + self-hosted stack
Infrastructure as Code, one codebase	✓ This repository

What's deferred, and why

Honest about scope, not silent about it:

- **Ghost horizontal scaling — ruled out**, not deferred: Ghost's own docs prohibit multi-instance clustering. Scale path is vertical resources + a future cache/CDN layer.
- **Backup/DR policy, GKE zonal vs. regional** — open questions, not yet analysed.
- **Hub-and-spoke networking** — considered, not needed at this scale; revisit only if hybrid on-prem connectivity becomes a real requirement.
- Full list with effort estimates: [docs/ROADMAP.md](#) .

Migration plan & next steps

- **Lowest-risk first:** Test migrates, then Acceptance, then Production — never in parallel.
- **Parallel run, not a big-bang switch:** on-prem stays live and untouched until the new environment is validated; DNS cutover is a five-minute, reversible action.
- **Soak period per environment** before moving to the next, and before decommissioning on-prem.

Full runbook: [docs/MIGRATION.md](#) .

Next step: review and sign-off to begin Phase 0 (target infrastructure readiness).